Iteration 1 — Change Report

Nav2 parameter and behaviour-tree changes from the four real course runs of 28 October 2026

Files changed (previous versions saved in `iterations/1/OldFiles/`):

File	What changed
imprimis_navigation/config/nav2/**Course2027.yaml**	Ray-trace clearing switched on in both cost maps; the global map shrunk from 100 m to 30 m square and both update rates cut; MPPI's speed and yaw envelope rebuilt; the three path critics' look-aheads corrected for the lattice's 2.5 cm path spacing; goal tolerance 1 m and heading-agnostic goals; a progress checker written down.
imprimis_navigation/nav2_bt/**recovery_nav_bt.xml**	Cost-map clearing re-enabled as the last two entries of the round robin only; replanning 1 Hz → 2 Hz; retries 6 → 8; waits lengthened to a full LiDAR sweep.

Everything added to the YAML is tagged `ITER1`, so `grep ITER1 Course2027.yaml` lists every change with its reasoning next to it. Nothing outside these two files was touched; the changes that are needed elsewhere are in section 6, and **one of them matters as much as anything in this report**.

The tools written to read the bags are in `claude_tools/` and are listed in section 7. Every number quoted below comes out of one of them, from the bags in `iterations/1/RealBags/`.

1. What the robot did

Runs 1, 3 and 4 stopped dead and never restarted; run 2 reached the goal. The bags agree with the written observations, and add the detail that all three failures are the same failure:

Run	Goal distance	Driven	Stopped	Top speed	Top yaw rate	Outcome
1	7.93 m	8.50 m	t = 14.6 s	0.50 m/s	0.200 rad/s	stopped 4.83 m short
2	7.29 m	9.00 m	t = 34.6 s	0.50 m/s	0.200 rad/s	**reached the goal** (0.27 m)
3	6.51 m	4.47 m	t = 21.7 s	0.50 m/s	0.200 rad/s	stopped 3.79 m short
4	7.65 m	6.96 m	t = 25.6 s	0.50 m/s	0.200 rad/s	stopped 5.51 m short

Three things in that table are worth staring at before anything else. Every run topped out at **exactly** 0.500000 m/s and **exactly** 0.200000 rad/s. And the one run that succeeded differs from the other three in no parameter at all — it is the same software, and it got lucky.

The shape of each failure is identical

In all three failures the sequence, to within one tree tick, is:

```
t      last /plan published
t+1.05 a plan was due and none appeared
t+1.10 BackUp starts: v = -0.250 m/s
t+1.45 BackUp stops after 0.07-0.10 m of the 0.6 m it was asked for
then   a single zero command every 1.06 s, forever
```

Run 1: last plan 13.15 s, back-up at 14.25 s. Run 3: 20.55 s, stop at 21.67 s. Run 4: 24.52 s, back-up at 25.61 s, one more attempt, then back-up again at 28.16 s and nothing after.

That ordering says the planner failed, not the controller. `/plan` is published by `ComputePathToPose` inside the `RateController`; the controller cannot stop it from publishing. A plan going missing, and the recovery branch firing on the same tick, means `ComputePathToPose` returned failure and the `PipelineSequence` failed

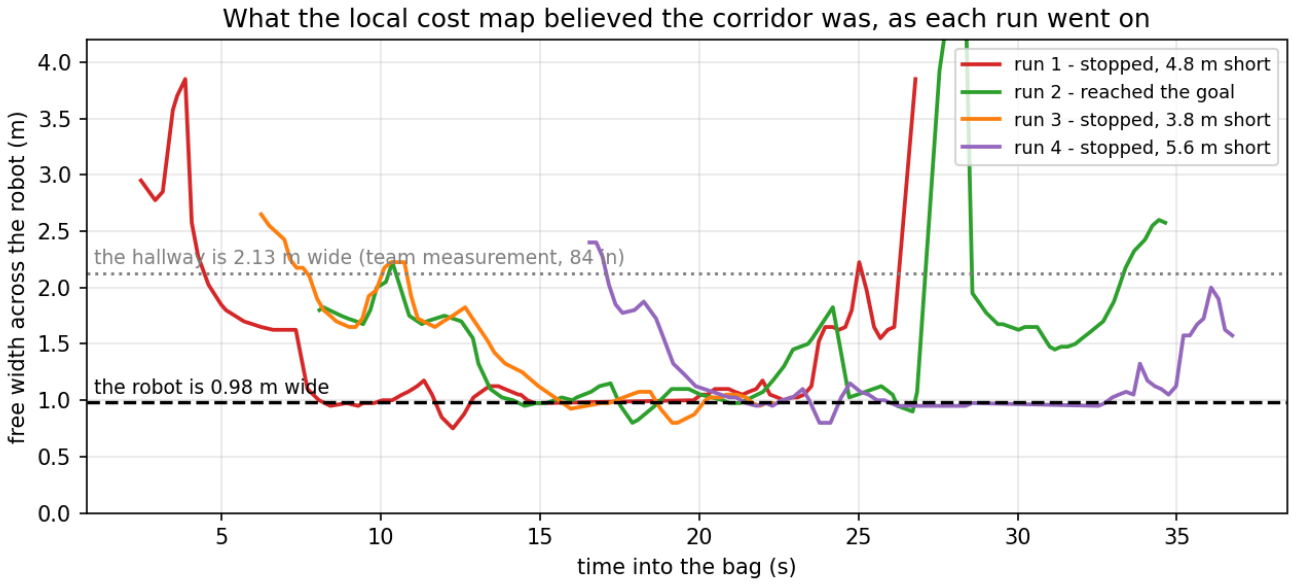
with it. The 1.06 s zeros afterwards are the behaviour server's stop command at the end of each round-robin entry.

We also checked the one planner failure that would have had a different cure: the robot's own centre cell was never at cost 253 or 254 in any frame of any run (it stayed 0.36–0.57 m from the nearest such cell, at cost 103–136). So this was not *StartOccupied* — the robot was not standing in an obstacle. The planner simply could not find a route through to the goal.

2. Why — the cost maps were closing in on the robot

Both cost maps had `clearing: False` on every observation source. With clearing off, the obstacle layer can only ever *add* cells. Nothing takes a mark away. A wall seen from a slightly different angle, a return off a doorway, a person walking past — all of it goes in and stays in for as long as the rolling window keeps that patch of ground.

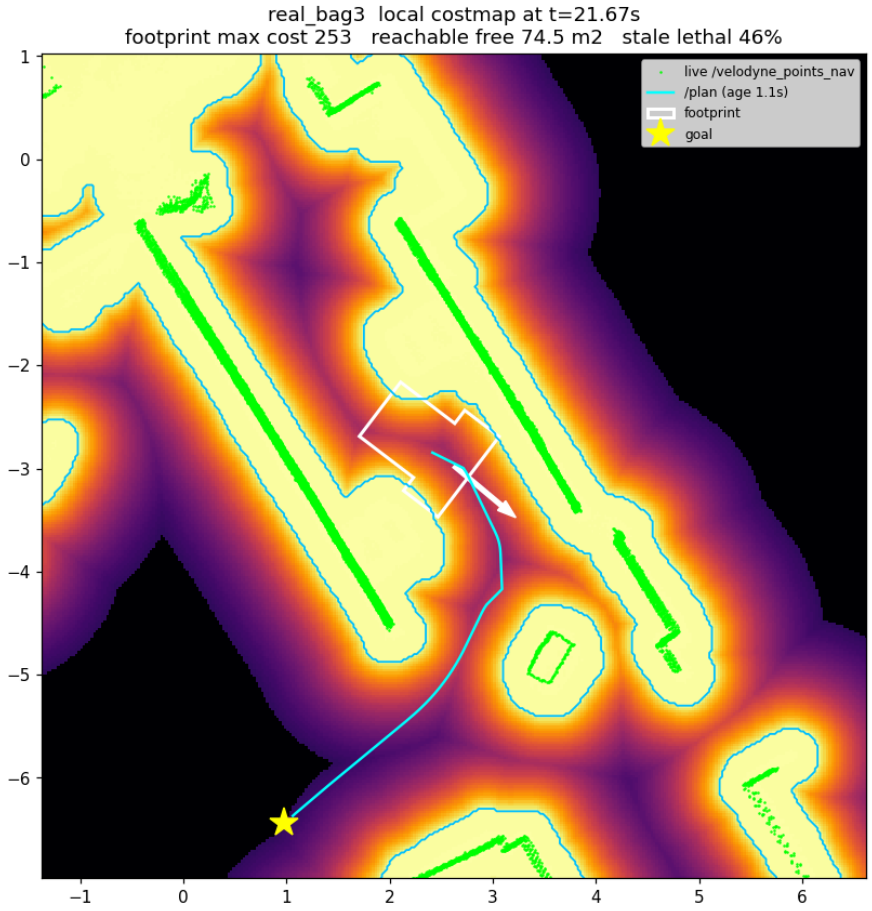
What that does over a thirty-second run:



Free (cost < 253) width measured across the robot's heading in the local cost map. Every run spends ten seconds or more pressed against the dashed line — the cost map claiming less room than the robot physically occupies — in a corridor the team measured at 2.13 m. The spikes at the right-hand end of each trace are the tester driving the robot back out into the open.

Three independent measurements of the same thing:

Measurement	Result
Lethal cells in the local cost map with no LiDAR return within 15 cm of them, at the moment the robot stopped	**35%, 57%, 46%, 46%** (runs 1–4). Between a third and a half of the obstacles the robot believed in were not there any more.
Free width across the robot, run 3, from the moment it entered the hallway	2.23 → 1.93 → 1.75 → 1.53 → 1.33 → 1.13 → 0.93 → **0.80 m** over twelve seconds, monotonic. The robot is 0.98 m wide.
Minimum free width across the robot, all four runs	**0.75, 0.80, 0.80, 0.80 m**. Not one run escaped it.
Fraction of the local cost map that was free at the stop	32% free, 7.5% lethal, 15% at 253, 45% inflated.



Run 3 at the moment it stopped. Bright = high cost. Green dots are the *live* LiDAR return at that instant; the blue outline is the boundary of the space the robot could still reach. The two long green lines are the real hallway walls. Everything bright that is not sitting on a green line is a mark the cost map kept after the obstacle had gone. The white outline is the robot: its own footprint is inside the 253 band.

This is the proximate cause of all three failures. The Smac lattice planner needs a chain of poses whose centre cell is below 253 and whose footprint outline clears 254. As the walls grew inward, that chain ran out. The run-2 trace in the figure above shows the same squeeze happening and then easing off again — which is exactly why run 2 succeeded and the others did not. It was margin, not design.

It also explains the observation written down for run 1, that the tester drove back through cells the cost map had marked as occupied without hitting anything. And it is why the deliberate “ghost obstacles” in run 4 worked so well: a person standing in front of the robot for two seconds was written into the map permanently.

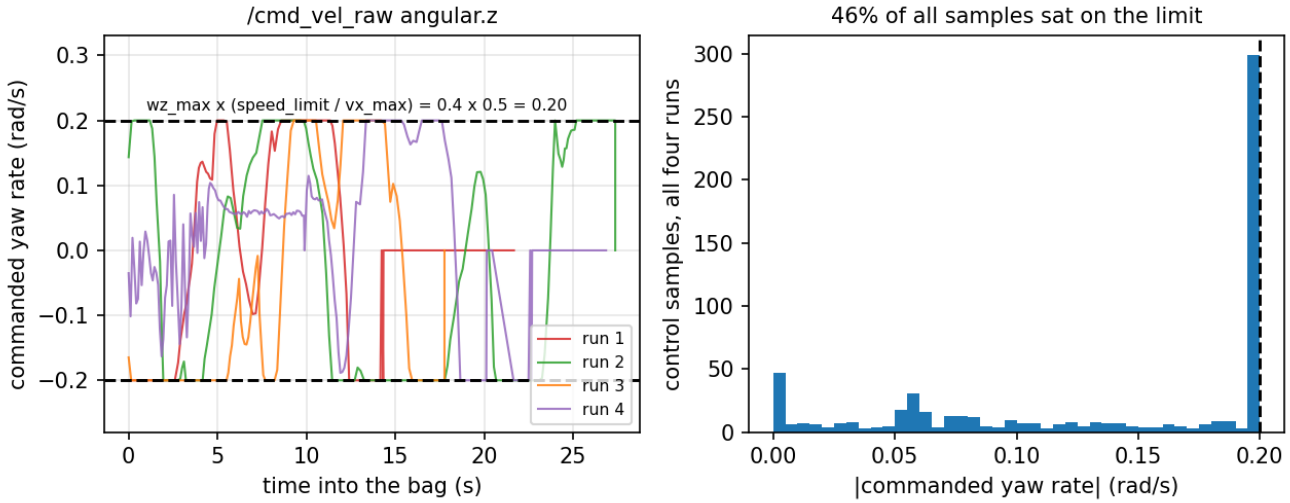
3. Why — the robot could only turn a quarter as hard as the path needed

Every command in every bag is capped at exactly 0.500000 m/s and 0.200000 rad/s. The speed is the speed governor doing its job. The yaw rate is an accident.

`speed_governor.py` publishes an **absolute** speed limit (`msg.percentage = False`). Nav2 hands that to MPPI, and `Optimizer::setSpeedLimit` handles the absolute case like this:

```
double ratio = speed_limit / s.base_constraints.vx_max;
s.constraints.vx_max = s.base_constraints.vx_max * ratio;
s.constraints.vx_min = s.base_constraints.vx_min * ratio;
s.constraints.wz     = s.base_constraints.wz     * ratio;
```

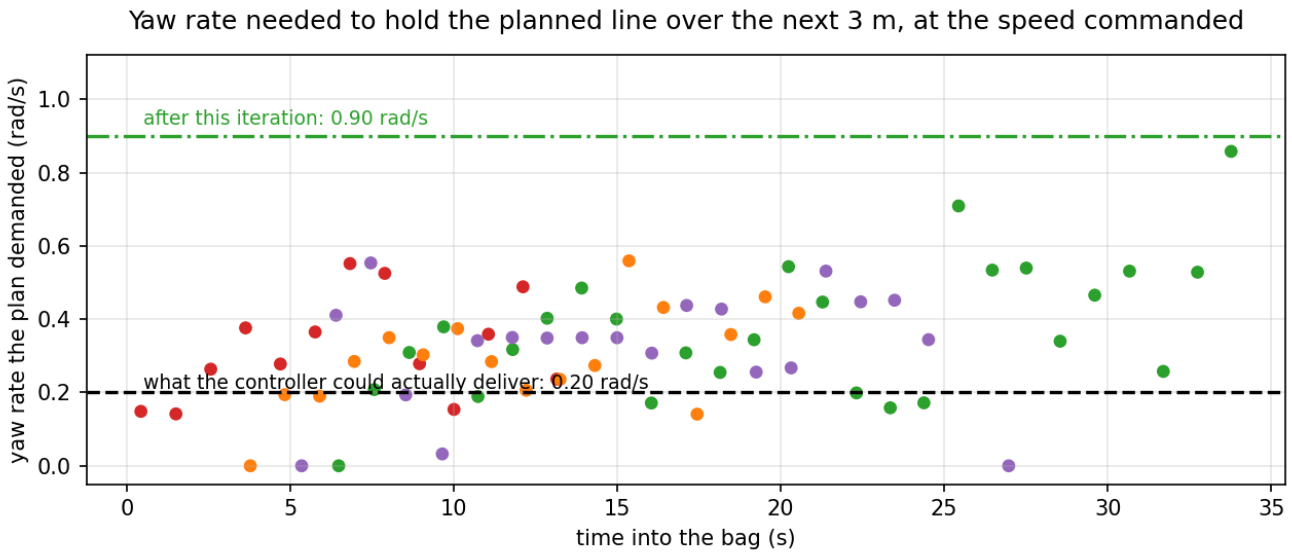
The same ratio is applied to the yaw rate. `nav_perception.launch.py` declares `top_speed` with a default of **0.5**, and `Course2027.yaml` had `vx_max: 1.0`, so ratio = 0.5 and the real yaw limit was 0.4 × 0.5 = **0.20 rad/s** — and the reverse limit was halved too, from −0.35 to −0.175 m/s.



Left: the commanded yaw rate in all four runs, against the 0.20 rad/s ceiling. Right: the same data as a histogram. The controller was asking for everything it was allowed, almost half the time.

Per run, the share of control samples sitting on the limit was **44%, 60%, 67% and 20%**. Meanwhile the linear velocity touched its own 0.5 m/s ceiling only 0–5% of the time. The robot was not speed-limited. It was **turn-limited**, badly.

How badly: 0.50 m/s with 0.20 rad/s is a minimum turning radius of **2.50 m**, in a hallway 2.13 m wide. The lattice in `diff_2.5cm.json` is built around a 0.5 m minimum radius and the plans it produced asked for 0.45–1.5 m radii.



For each published plan, the yaw rate needed to hold its line over the next 3 m at the speed the robot was being commanded. 76–77% of planning cycles asked for more than the controller could give. Curvature is measured over a 0.75 m chord, because at the lattice's 2.5 cm point spacing a shorter baseline measures pose noise rather than the shape of the path.

The consequence compounds with section 2. A robot that cannot hold the planned line drifts toward a wall; sitting nearer a wall, it sees it from a new angle and marks a few more cells; those cells are never cleared. The corridor closes a little faster, the plan gets tighter, the robot falls further behind it. **Run 2 is the same loop that happened to unwind in time.**

4. Why — the recovery could not undo any of it

Every `ClearEntireCostmap` in `recovery_nav_bt.xml` was commented out, which left the round robin as **BackUp, Wait, BackUp, Wait**. Against a failure caused entirely by stale marks, none of those four actions can help, and two of them could not even run:

The behaviour server refuses a BackUp whose projected footprint covers a lethal cell (`simulate_ahead_time: 1.0`, so it checks 0.25 m behind). The robot's tail reaches 0.86 m behind the drive axle, straight into the band of cells the cost map wrongly believed were lethal. Measured from the bags: run 1's BackUp covered **0.10 m** of the 0.6 m asked for before aborting; run 4's covered **0.075 m**.

So the robot sat in a cost map of its own making, with the one action that would have freed it switched off, until the retries ran out and `NavigateToPose` aborted.

A note on the thing that is not the cause

We checked and ruled out several plausible alternatives rather than assuming them, because each would have led to a different fix:

Hypothesis	Verdict
MPPI ran out of collision-free rollouts (*NoValidControl*)	**No.** Replaying MPPI's own collision rule over a grid of rollouts at each stop state, 62–81% of them survive. Survivors existed.
The planner's start pose was occupied (*StartOccupied*)	**No.** The robot's centre cell never reached 253 in any frame of any run.
`PathAlignCritic` was switching itself off via `max_path_occupancy_ratio`	**No.** 0% of plan points had a centre cell at ≥253 — the planner guarantees that, so the critic stayed active throughout.
The global and local cost maps disagreed, because they read different LiDAR topics	**Not here, but it is a live hazard.** Indoors the two agreed to within 0.03 m of corridor width. The global map was reading the raw `/velodyne_points` (ground included, not IMU-levelled) rather than the filtered cloud its own comment said it should read. On the ramp they would not have agreed. Fixed anyway.

And the one the team already suspected

The controller server misses its deadline. `/cmd_vel_raw` came out at **6.8, 6.7, 7.6 and 9.9 Hz** against `controller_frequency: 10.0`. The local cost map managed 15–22 Hz of its 30 Hz target and published at 1.5–2.7 Hz against a 5 Hz setting. Two things were eating the time:

• The global cost map was **100 × 100 m at 0.025 m — 16 million cells — rebuilt and re-centred 30 times a second**, with a 3 m inflation radius (a 240-cell-wide kernel). It is a rolling window, so every update copies the whole grid. The waypoints in these tests were 6.5–7.9 m away.
• MPPI ran **20000 rollouts × 20 steps with a full footprint check at every point** (`trajectory_point_step: 1`), in a cost map that was 68% non-free, so most points triggered one.

The team's instinct that 10 Hz is plenty for a robot this slow is right, and this iteration acts on it.

5. The changes

5a. The cost maps — this is the one that matters

Parameter	Was	Now	Why
`obstacle_layer.velodyne_data.clearing` (both maps)	False	**True**	Ray-trace clearing. A mark is removed once the LiDAR sees through where it used to be. Everything in section 2 follows from this being off.
`global … velodyne_data.topic`	/velodyne_points	/velodyne_points_nav	The filtered, IMU-levelled, ground-removed cloud that the local map already used, and that the line's own comment said it should be.
`global width / height`	100 m	30 m	16 M cells → 1.44 M. Still ±15 m around the robot, twice the longest leg driven.
`global update_frequency`	30.0	5.0	The tree replans at 2 Hz.
`local update_frequency`	30.0	10.0	Matches `controller_frequency`, which is how often the map is read. It was only reaching 15–22 Hz anyway, by stealing time from MPPI in the same process.
`publish_frequency` (both)	5.0	2.0	These topics feed RViz and the speed governor; nothing in the planning loop reads them.
`global inflation_radius`	3.0	2.0	Still far above the 0.92 m furthest footprint corner. At 3 m every cell in a 2.1 m corridor is inside the kernel from both walls at once, which flattens the gradient we want.
`raytrace / obstacle max_range`	6.0 / 6.0	9.0 / 8.0 global 7.0 / 6.0 local	Ray-trace reach kept above marking reach, so the far edge of a marked region can be cleared too.
`global layer obstacle heights`	0.3 – 1.8	0.2 – 3.0	They disagreed with the source below them and with the local map. Harmonised.

5b. The speed and yaw envelope

Parameter	Was	Now	Why
`vx_max`	1.0	**0.5**	Set equal to the governor's own top speed, so ratio = 1.0 and the yaw rate stops being halved. **Read section 6.1 before changing either number.**
`wz_max`	0.4	**0.9**	0.9 rad/s at 0.5 m/s is a 0.56 m turning radius, which covers the 0.5 m minimum the lattice is built around and the 0.45–1.5 m the plans actually asked for.
`vx_std / wz_std`	0.4 / 0.4	0.2 / 0.45	Sampling spread matched to the new ranges; it was spending most of a 20000-sample batch on values that get clamped.
`ax_max / ax_min`	±3.0	±1.5	Measured from the bags: the robot reaches 0.30–0.56 m/s² (p95) and peaks at 1.4. MPPI rolls out trajectories using these, so optimistic values make it plan motions the robot never executes.
`az_max`	3.5	2.0	Measured peak 1.7 rad/s²; the smoother caps it at 2.0 regardless.
`vx_min`	–0.35	–0.3	Matches the 0.25 m/s the BackUp recovery asks for.

Parameter	Was	Now	Why
`velocity_smoother max/min_velocity`	±[2.2, 0, 1.5]	±[2.235, 0, 1.2]	2.235 m/s is exactly 5 mph. This is now a deliberate backstop: because MPPI does *not* clamp the speed-limit ratio to 1, a `top_speed` above `vx_max` would scale `wz` up without bound. These two caps hold whatever the governor asks for.
`smoother max_accel / max_decel`	±[3.0, 0, 2.0]	[1.5, 0, 2.0] / [–2.0, 0, –2.5]	Matched to MPPI's limits; braking allowed to be firmer than accelerating.

5c. The path critics — a units problem

`offset_from_furthest` counts **path points, not metres**. The Smac lattice hands us a path with **2.5–2.8 cm between points** (measured: 390 poses over 9.96 m). The values in the file were Nav2's defaults, which assume the 5–10 cm spacing of the grid planners, so every look-ahead was a small fraction of what it was meant to be. `PathFollowCritic` was the worst affected: it drags the end of each rollout toward the path point `offset_from_furthest` beyond the furthest point the rollouts already reach, and at 5 points that carrot was **13 cm** out — essentially no forward pull at all.

Critic	Parameter	Was	Now	In metres of path
PathFollowCritic	offset_from_furthest	5	24	0.13 m → 0.61 m
PathAlignCritic	offset_from_furthest	5	20	0.13 m → 0.51 m
PathAngleCritic	offset_from_furthest	10	20	0.26 m → 0.51 m
PathAngleCritic	max_angle_to_furthest	1.0	0.6	starts lining up at 34° of error instead of 57°, so it enters a turn already pointing into it

5d. Staying away from the walls, and stopping at the goal

Parameter	Was	Now	Why
`goal_checker.xy_goal_tolerance`	0.3	**1.0**	“Stop as soon as it is within 1 m of the waypoint.”
`goal_checker.yaw_goal_tolerance`	6.28	6.28	Unchanged — already accepts any heading.
`GridBased.goal_heading_mode`	DEFAULT	**ALL_DIRECTION**	DEFAULT makes the planner arrive on the goal's exact heading, which is the expensive detour we did not want. ALL_DIRECTION lets it finish on whichever of the headings is cheapest. `coarse_search_resolution: 2` tries 8 of the 16 rather than all of them.
`GoalAngleCritic`	in the list	**removed**	It pulls the robot toward the goal's heading on the way in. We do not care what the heading is. Its block is left in the file, disabled, in case a heading-sensitive goal is ever wanted.
`GridBased.tolerance`	2.0	1.0	Matched to the goal tolerance, so a plan that ends short still ends inside the distance at which the robot stops. At 2.0 the robot could reach the end of its path 2 m out and never trip the goal checker.
`CostCritic.cost_weight`	3.81	5.0	The cost this critic reads *is* the inflation field, so weighting it harder is the direct way to ask for “as far from the walls as the path allows”.
`GridBased.cost_penalty`	2.0	3.0	The same request, on the planner's side.
`GridBased.rotation_penalty`	0.0	**5.0**	`diff_2.5cm.json` contains in-place rotation primitives (`trajectory_length 0.000`) and at 0.0 the planner could use them for free. The tail sweeps a 1.8 m circle — which is why Spin is kept out of the recovery tree. The planner should not be allowed them either.

`GoalCritic.threshold_to_consider`
`PathFollow/Align/Angle thresholds`
`CostCritic.near_goal_distance`
	1.0
	1.4 / 0.5 / 0.5
	1.0
		1.5
		1.5 / 1.2 / 1.2
		1.5
			All of these hand over from path-following to goal-seeking at a fixed distance from the goal. With the stop radius now 1.0 m they all have to move out past it, or the hand-over never happens before the robot has already stopped.

5e. Making the controller keep up

Parameter	Was	Now	Why
`batch_size`	20000	6000	With the step change below, about 6× less work per control cycle. 6000 is still three times Nav2's default.
`CostCritic.trajectory_point_step` `PathAlignCritic.trajectory_point_step`	1	2	At 0.5 m/s and `model_dt 0.1` the trajectory points are 5 cm apart, so every second one is a 10 cm check interval on a robot 1.08 m long. Finer than the cost map's own cells can resolve, and half the footprint checks.
`time_steps`	20	30	20 × 0.1 s × 0.5 m/s is 1.0 m of look-ahead — *shorter than the robot*. 30 gives 3.0 s / 1.5 m, so a rollout can see past its own tail. This is an increase in work, deliberately spent where it helps.
`prune_distance`	5.0	2.5	At 0.5 m/s a 5 m window is ten seconds of path, and its far end falls outside the 10 m local cost map, where MPPI counts the points as invalid.
`max_planning_time`	5.0	2.0	Planning measured under 0.1 s in every run. A 5 s budget only delays the failure report.
`failure_tolerance`	(absent, 0.0)	0.3	One bad control cycle no longer drops the whole tree into recovery.
`progress_checker`	(absent)	written down	It was running all along on Nav2's defaults — **0.5 m of movement every 10 s** — which nobody had written down and which is tight for a robot creeping past a bin at 0.5 m/s. Now explicit, at 0.35 m every 20 s.

5f. The behaviour tree

The old round robin was **BackUp(0.6)** → **Wait(1)** → **BackUp(0.8)** → **Wait(1)**, with all three clearing blocks commented out. The new one is:

#	Action	Why there
1	`BackUp 0.5 m @ 0.25 m/s`	0.5 m rather than 0.6: the shorter the reverse, the better its chance of passing the behaviour server's own collision check and actually completing.
2	`Wait 2.0 s`	Long enough for several full Velodyne sweeps at 10 Hz to mark *and, now, to clear*. The old 1.0 s was not.
3	`BackUp 0.8 m @ 0.25 m/s`	Unchanged in intent.
4	`Wait 2.0 s`	
5	**Clear the LOCAL cost map**, then Wait 2.0 s	**First resort of the last resorts.** The global map keeps the route, so the robot cannot wander off it while the controller re-reads the space immediately around itself.
6	**Clear BOTH cost maps**, then Wait 3.0 s	**True last resort.** Everything is rebuilt from the sensors. The wall layer is not clearable, so the virtual wall at the start line survives this.

Spin stays out — the reasoning in the file's header is right, and nothing in these bags argues against it. Other changes: `RateController hz` 1.0 → **2.0** (the cost maps now change meaningfully between ticks because they clear as well as mark, and the probes found the controller working from a 1.12 s old plan — half a metre of travel); `number_of_retries` 6 → **8**, so that there are navigation attempts left *after* the clearing steps rather than the clearing being the last thing that ever happens.

The `FollowPath` context branch is deliberately left with no clear in it. A single bad control cycle is not a reason to throw away what the cost maps know; `failure_tolerance` absorbs those, and anything worse falls through to the round robin.

6. Changes outside these two files — for the team to make

6.1 Raise top_speed, and raise vx_max with it, in the same commit [important]

`imprimis_navigation/launch/nav_perception.launch.py`, line 26:

```
DeclareLaunchArgument("top_speed", default_value="0.5", …)
```

0.5 m/s is 1.1 mph. The IGVC limit is 5 mph. If the robot is meant to run faster than a walking pace, this is the number to change — but **it must move together with** `FollowPath.vx_max` in Course2027.yaml, because MPPI derives the yaw limit from their ratio and does not clamp it to 1:

`top_speed`	`vx_max`	Resulting top speed	Resulting yaw limit	
0.5	1.0	0.50 m/s	0.20 rad/s	**what ran in these tests** — 2.5 m turning radius
0.5	0.5	0.50 m/s	0.90 rad/s	**what is set now** — 0.56 m turning radius
1.5	1.5	1.50 m/s	0.90 rad/s	a sensible next step, once the cost maps are trusted
2.1	2.1	2.10 m/s	0.90 rad/s	4.7 mph, inside the rule
2.1	0.5	2.10 m/s	**3.78 rad/s**	**the trap.** Raising one without the other. The velocity smoother clamps the wheels to 1.2 rad/s, but MPPI's rollouts still believe in 3.78 and plan accordingly.

The governor's own floor is `max(0.4, …)`, so in the tightest gap the ratio bottoms out at 0.4/top_speed. At `top_speed 2.1` that is a yaw limit of 0.17 rad/s in exactly the places that need the most turning. **Consider making the governor publish a percentage** (`msg.percentage = True`) and clamping its floor as a fraction, which removes the coupling to `vx_max` altogether. Alternatively raise `tight_speed` (currently 0.9) and the 0.4 floor together.

6.2 Record the topics that would have made this diagnosis trivial

Add to the bag recording for iteration 2:

Topic	Why
`/rosout`	**The single most valuable addition.** Every conclusion in section 1 about *which* server failed had to be inferred from publication timing. The planner logs its own error (*NoValidPathCouldBeFound*, *StartOccupied*, *PlannerTimedOut*) and would have said so outright. It would also show the “control loop missed its desired rate” warnings.
`/behavior_tree_log`	Which node ticked, when, and what it returned. Turns section 4 from reconstruction into a transcript.
`/speed_limit`, `/speed_governor/info`	The governor already publishes its limit *and its reason* as JSON. Section 3 would have been a one-line read.
`/local_costmap/costmap_raw`, `/global_costmap/costmap_raw`	Raw 0–255 costs. `/…/costmap` is the OccupancyGrid, rescaled to 0–100, so 253 and 254 have to be reconstructed and the distinction between them — which decides whether the planner rejects a pose — is the whole game.
`/plan_smoothed`, `/local_plan`, `/trajectories`	What MPPI was actually considering. Only needed if a controller problem is suspected; they are large.
`/velodyne_points` (raw)	Only the filtered cloud was recorded, so the global cost map's input in these runs could not be checked directly — it had to be inferred from the two maps agreeing.

6.3 Other things worth doing, none of them changed here

	Observation
Lane and pothole points have the same trap	`lane_data` and `pothole_data` are still `clearing: False`, and they are left that way on purpose: ray-tracing them from the camera would rub out a lane line as the robot drives past it. But they will accumulate exactly as the LiDAR marks did, and on the real course a lane line is a wall. Before the lane branch goes live, `lane_mapper` needs to publish points with a limited lifetime, or those layers need a decay. **Worth testing deliberately before it is found on the course.**
The 1.06 s zero-velocity pulses	While the round robin cycles, the only thing reaching the wheels is one zero command per second. The robot looks dead to anyone watching. A status LED or a line in the control window driven by `/behavior_tree_log` would save the testers a lot of guessing about whether it has given up.
`lattice_filepath` is an absolute path	`/home/imprimis-pc/Desktop/imprimis/…` in Course2027.yaml. It works on this laptop and nowhere else. A `$(find-pkg-share …)` substitution, like the `default_nav_to_pose_bt_xml` line already uses, would survive a fresh clone.
Cost map resolution	0.025 m is fine and we have **not** changed it — the lattice is generated for it, and the global map shrinking from 100 m to 30 m is an 11× saving where a resolution change would have been 4×. If more headroom is wanted later, regenerate the lattice at 0.05 m with Lattice_Generator and halve both maps' resolution together. Do not change one without the other.
Run 2 was not a different configuration	It is worth saying plainly, because it is easy to read the four runs as “three failures and a success”. All four ran identical software. Run 2 is one sample of the same failing process that happened to clear the hallway before the cost map closed. Treat it as a near miss, not as evidence that the configuration worked.

7. What to look for in iteration 2, and the tools

The changes in sections 5a and 5b should be visible in the next set of bags without any analysis at all, and these are the numbers that will say whether they worked:

Check	Iteration 1	Target
Minimum free width across the robot (`corridor_trend.py`)	0.75–0.80 m in all four runs	stays above 1.4 m in the 2.13 m hallway
Lethal cells with no live LiDAR support (`costmap_vs_lidar.py`)	35–57%	under 10%
Share of control samples at the yaw limit (`cmd_limits.py`)	20–67%	under 15%
Controller rate (`bag_summary.py`, `/cmd_vel_raw`)	6.7–7.6 Hz	9.5–10 Hz
Planning cycles demanding more yaw than available (`curvature_check.py`)	76–77%	under 10%
Recovery entries reached (`endgame.py`)	BackUp and Wait only, both BackUps aborting	if recovery fires at all, a BackUp that completes its full distance

If the robot still stops, the first thing to check is whether a cost-map clear in the round robin got it moving again. If it did, the marks are still going stale and the source is upstream of Nav2 (`ramp_detector`, or the lane points of 6.3). If clearing does *not* help, the obstacle is real and the question becomes whether the gap is genuinely too tight — which is a footprint and inflation question, not a clearing one.

The tools, in claude_tools/

Tool	What it does
`bagread.py`	Shared reader. `read_topics(bag, [topics])` yields (topic, t, msg).
`bag_summary.py`	Topics, counts and effective rates from metadata alone. Instant.
`run_timeline.py`	Per-run 1 Hz table of pose, speed, commands and distance to goal, plus every plan and every gap in the controller output. Start here.
`endgame.py`	Message-by-message log of the last seconds, which tells a controller command from a BackUp.
`cmd_limits.py`	Finds hard caps in the commanded velocity. This is what exposed the 0.200000 rad/s ceiling.
`costmap_probe.py`	ASCII dump of either cost map around the robot at one instant, with footprint cost, corridor width and the plan overlaid.
`costmap_vs_lidar.py`	The picture in section 2: cost map against the live LiDAR, with the stale-mark percentage and the reachable free area.
`clearance_audit.py`	Whole-run sweep of footprint cost, corridor width and the measured inflation profile.
`map_compare.py`	Global against local cost map, frame by frame.
`curvature_check.py`	Yaw rate each plan demands versus what the controller can give.
`mppi_replay.py`	Replays MPPI's collision rule over a grid of rollouts, so the effect of a limit change can be seen before a test run.
`start_occupied.py`, `plan_feasibility.py`, `path_validity.py`	The three checks that ruled out the wrong explanations in section 4.
`dynamics.py`	Achieved acceleration and command tracking error. Source of the ax/az numbers.

Tool	What it does
`corridor_trend.py`, `make_figures.py`	The figures in this report.

All of them take bag directories as arguments and need `source /opt/ros/kilted/setup.bash` first. One caveat on the corridor-width figure: it is measured perpendicular to the robot's heading, so a robot sitting at an angle across the corridor reads a little narrower than the corridor really is. The ten-second stretches at 1.0 m are not that — they are the cost map.